Java vs TypeScript

A Java developer's quick reference | Mathieu Néron | 2026

This cheatsheet anchors TypeScript on constructs you already know in Java. Left column: Java. Right column: TypeScript (5.x). The biggest mental shifts: **structural typing** (shape, not name), **single-threaded event loop** with `async`/`await`, **union/literal types**, and a vast **type system that erases at runtime**.

1. Hello world & program entry

Program entry point

Java

```java
public class Hello {
  public static void main(String[] args) {
    System.out.println("Hello, " + args[0]);
  }
}
// javac Hello.java && java Hello world
```

TypeScript

```typescript
// hello.ts (Node 20+ / Bun / Deno)
const name: string = process.argv[2];
console.log(`Hello, ${name}`);

// $ tsx hello.ts world      (or: tsc && node hello.js)
// In a module, top-level code just runs at load.
```

File / module / package layout

Java

```java
// One public class per file.
package com.acme;
// pom.xml / build.gradle declares deps.
```

TypeScript

```typescript
// Each .ts file is a module if it has import/export.
// package.json + node_modules. Entry: "main" or "exports".
// Scripts via "scripts": { ... }; pnpm/npm/bun.
// tsconfig.json controls compiler.
```

2. Variables, constants, type inference

Local variables

Java

```java
int x = 5;
final String s = "hi";
var list = new ArrayList<Integer>();   // Java 10+
```

TypeScript

```typescript
let x: number = 5;            // mutable
const s: string = "hi";       // immutable BINDING
const list: number[] = [];    // type can be inferred
const arr = [1, 2, 3];        // inferred number[]
// `var` exists but avoid it - function-scoped, hoisted.
```

Const != deeply immutable

Java

```java
final List<Integer> xs = new ArrayList<>(List.of(1,2));
xs.add(3);                  // OK - final binding only
List<Integer> immut = List.of(1, 2);
immut.add(3);               // throws UnsupportedOp
```

TypeScript

```typescript
const xs = [1, 2];
xs.push(3);                 // OK - const binding only
const immut: readonly number[] = [1, 2];
immut.push(3);              // type error
const tuple = [1, 2] as const;   // readonly tuple [1,2]
```

3. Primitive types & conversions

Numeric & boolean types

Java

```java
byte/short/int/long; float/double;
int n = Integer.parseInt("42");
double x = Double.parseDouble("1.5");
String t = String.valueOf(42);
```

TypeScript

```typescript
// number = 64-bit IEEE float (no separate int).
// bigint = arbitrary precision (literal: 42n).
const n: number = parseInt("42", 10);
const x: number = parseFloat("1.5");
const t: string = String(42);
const big: bigint = 42n;
```

Strings (template literals)

Java

```java
String name = "world";
String s1 = "Hello, " + name;
String s2 = String.format("%s = %.2f", "pi", 3.14);
String body = """
    Hello,
    World
    """;
```

TypeScript

```typescript
const name = "world";
const s1 = `Hello, ${name}`;
const s2 = `${"pi"} = ${(3.14).toFixed(2)}`;
const body = `
    Hello,
    World
    `;
// Tagged templates: fn`a ${x} b` -> fn(strings, x)
```

4. Control flow

If / else / ternary / nullish

Java

```java
if (x > 0) { ... } else if (x == 0) { ... } else { ... }
int sign = x > 0 ? 1 : (x < 0 ? -1 : 0);

String name = (s != null) ? s : "default";
```

TypeScript

```typescript
if (x > 0) { ... } else if (x === 0) { ... } else { ... }
const sign = x > 0 ? 1 : (x < 0 ? -1 : 0);

const name = s ?? "default";        // nullish-coalescing
const name2 = s || "default";       // FALSY (incl. "", 0)
const len = obj?.user?.name?.length;// optional chaining
```

Switch / discriminated unions

Java

```java
String label = switch (shape) {
  case Circle c       -> "circle r=" + c.r();
  case Rectangle(var w, var h) -> "rect " + w + "x" + h;
  default             -> "unknown";
};
```

TypeScript

```typescript
type Shape =
  | { kind: "circle"; r: number }
  | { kind: "rect"; w: number; h: number };

function label(s: Shape): string {
  switch (s.kind) {
    case "circle": return `circle r=${s.r}`;
    case "rect":   return `rect ${s.w}x${s.h}`;
  }
  // Exhaustiveness: TS narrows `s` to never above.
}
```

5. Loops

Counting / iteration

Java

```java
for (int i = 0; i < 10; i++) { ... }
for (var x : xs) { ... }
for (var e : map.entrySet()) {
    System.out.println(e.getKey() + "=" + e.getValue());
}
```

TypeScript

```typescript
for (let i = 0; i < 10; i++) { ... }

for (const x of xs)        { ... }      // values
for (const [i, x] of xs.entries()) { ... }
for (const [k, v] of m) { ... }         // Map
for (const k in obj) { ... }            // KEYS (rarely)

xs.forEach((x, i) => { ... });          // higher-order
```

Async iteration

Java

```java
// Reactive stack or a custom Iterator. No syntax sugar.
```

TypeScript

```typescript
for await (const chunk of stream) {     // AsyncIterable
    process(chunk);
}
```

6. Collections & data structures

JavaScript's built-ins are: `Array`, `Map`, `Set`, `WeakMap`, `WeakSet`, plus typed-array buffers. Both `Map` and `Set` preserve insertion order — so they double as `LinkedHashMap`/`LinkedHashSet` out of the box. For sorted maps, priority queues, and deques you'll reach for npm libraries (`mnemonist`, `js-sdsl`, or `sorted-btree`).

Quick reference

Java	Behavior	TypeScript
`ArrayList<T>`	dynamic array	`T[]` (`Array<T>`)
`LinkedList<T>`	doubly-linked	`mnemonist/linked-list`; or just use Array
`int[] / Object[]`	fixed-size array	`Int32Array(N)` typed array; `Array(N)` (sparse)
`HashSet<T>`	unordered set	`Set<T>` (note: insertion-ORDERED)
`LinkedHashSet<T>`	insertion-ordered set	`Set<T>` (it already is)
`TreeSet<T>`	sorted set	`js-sdsl OrderedSet`; `sorted-btree`
`EnumSet`	bit-set over enum	`Set<MyEnum>`
`HashMap<K,V>`	unordered map	`Map<K,V>` (insertion-ORDERED); object-as-dict for string keys
`LinkedHashMap<K,V>`	insertion-ordered	`Map<K,V>` (it already is)
`TreeMap<K,V>`	sorted map	`js-sdsl OrderedMap`; `sorted-btree`
`IdentityHashMap`	identity-keyed	`Map<K,V>`: object keys ARE compared by reference
`EnumMap`	enum-keyed map	`Map` or `Record<MyEnum, V>`
`WeakHashMap`	weak keys	`WeakMap<K,V>` (keys must be objects)
`ArrayDeque`	double-ended queue	`mnemonist/deque`; Array (shift is O(n))
`ArrayDeque (queue)`	FIFO	`mnemonist/deque` or `Array.shift` (slow)
`ArrayDeque (stack)`	LIFO	`Array.push/pop`
`PriorityQueue<T>`	min-heap	`mnemonist/heap`, `js-sdsl PriorityQueue`
`ConcurrentHashMap`	thread-safe map	N/A — single-threaded event loop; cross-worker via `postMessage`
`BlockingQueue`	blocking FIFO	async generator + `await`; or `node:stream`
`Counter`	multiset / tally	`Map<K,number>`; `Object.groupBy` (ES2024)
`List.copyOf, Map.of`	immutable view	`readonly T[]`, `ReadonlyMap<K,V>` (compile-time only); `Object.freeze` for runtime

Arrays, sets, maps (the natives)

Array (ArrayList analog)

Java

```java
List<Integer> xs = new ArrayList<>(List.of(1,2,3));
xs.add(4); xs.get(0); xs.size(); xs.remove(0);
xs.subList(1, 3); Collections.reverse(xs);
```

TypeScript

```typescript
const xs: number[] = [1, 2, 3];
xs.push(4); xs[0]; xs.length; xs.splice(0, 1);
xs.slice(1, 3);              // copy [1,3)
xs.toReversed();             // ES2023 (or [...xs].reverse())

// Higher-order:
xs.filter(x => x > 0).map(x => x*2);
xs.reduce((a, x) => a + x, 0);
xs.find(x => x === 2); xs.includes(2); xs.at(-1);

// Typed arrays (ArrayBuffer-backed, no boxing):
const buf = new Int32Array(1024);
```

Map (LinkedHashMap by default)

Java

```java
Map<String,Integer> m = new HashMap<>();
m.put("a", 1); m.getOrDefault("b", 0);
m.computeIfAbsent("c", k -> heavy());
m.merge("a", 1, Integer::sum);
```

TypeScript

```typescript
const m = new Map<string, number>();
m.set("a", 1); m.get("a") ?? 0;
m.has("a"); m.delete("a"); m.size;

// computeIfAbsent idiom:
let v = m.get("c") ?? (() => { const x = heavy(); m.set("c", x);
 ↪ return x; })();

// merge / tally:
m.set("a", (m.get("a") ?? 0) + 1);

// Iteration order = insertion order (always).
for (const [k, v] of m) { ... }

// Map vs object: Map keys can be ANY type. Object keys are
// always strings (or symbols). Map has .size; objects don't.
const r: Record<string, number> = { a: 1 };
```

Set (LinkedHashSet by default)

Java

```java
Set<String> hs = new HashSet<>();
Set<String> ls = new LinkedHashSet<>();
Set<String> ts = new TreeSet<>();
```

TypeScript

```typescript
const s = new Set<string>(["a", "b"]);
s.add("c"); s.has("a"); s.delete("b"); s.size;

// Set ops (ES2025):
s.union(t); s.intersection(t); s.difference(t);
s.symmetricDifference(t); s.isSubsetOf(t);

// Pre-ES2025 polyfill:
const u = new Set([...s, ...t]);                  // union
const i = new Set([...s].filter(x => t.has(x)));  // inter
```

Weak collections

WeakMap / WeakSet

Java

```java
Map<Key,V> wk = new WeakHashMap<>();
```

TypeScript

```typescript
const wm = new WeakMap<object, V>();      // keys MUST be objects
wm.set(keyObj, value);
wm.get(keyObj); wm.has(keyObj); wm.delete(keyObj);
// No iteration, no .size --- entries vanish silently when
// the key is GC'd. Useful for attaching metadata to objects
// without preventing collection.

const ws = new WeakSet<object>();
```

Sorted / priority structures (use libraries)

TreeMap / TreeSet (sorted)

Java

```java
TreeMap<String,Integer> tm = new TreeMap<>();
tm.firstKey(); tm.headMap("m"); tm.floorKey("g");

TreeSet<Integer> ts = new TreeSet<>();
ts.first(); ts.subSet(1, 10);
```

TypeScript

```typescript
// No stdlib. Pick a library:
import { OrderedMap, OrderedSet } from "js-sdsl";
const tm = new OrderedMap<string, number>();
tm.setElement("b", 2); tm.setElement("a", 1);
tm.front()!.first;       // smallest key

// Or sorted-btree:
import BTree from "sorted-btree";
const t = new BTree<string, number>();
```

PriorityQueue (heap)

Java

```java
PriorityQueue<Job> pq = new PriorityQueue<>(
    Comparator.comparingInt(Job::priority));
pq.offer(j); Job next = pq.poll();
```

TypeScript

```typescript
import Heap from "mnemonist/heap";
const pq = new Heap<Job>((a, b) => a.priority - b.priority);
pq.push(j); const next = pq.pop();

// Or js-sdsl PriorityQueue, or hand-roll an array-based
// binary heap (~30 lines) if you want zero deps.
```

Deque, queue, stack

Stack and queue patterns

Java

```java
Deque<Integer> st = new ArrayDeque<>();     // stack
st.push(1); st.peek(); st.pop();

Deque<Integer> q = new ArrayDeque<>();      // FIFO
q.offer(1); q.poll();
```

TypeScript

```typescript
// Stack: just an array.
const st: number[] = [];
st.push(1); const top = st.at(-1); const v = st.pop();

// Queue: Array.shift() works but is O(n) per call.
// For hot paths use mnemonist/deque (O(1) both ends):
import Deque from "mnemonist/deque";
const q = new Deque<number>();
q.push(1); q.shift();          // O(1)
```

Specialized

Counter / groupBy / immutable views

Java

```java
Map<String,Long> tally = words.stream()
    .collect(Collectors.groupingBy(w -> w, Collectors.counting()));
List<Integer> immut = List.copyOf(xs);
```

TypeScript

```typescript
// Tally:
const counts = new Map<string, number>();
for (const w of words) counts.set(w, (counts.get(w) ?? 0) + 1);

// ES2024 group-by (returns plain object / Map):
const byLen = Object.groupBy(words, w => w.length);
const byLenMap = Map.groupBy(words, w => w.length);

// Compile-time read-only (erased at runtime):
const xs: readonly number[] = [1,2,3];
const m: ReadonlyMap<string, number> = new Map();

// Runtime freeze (shallow):
const frozen = Object.freeze({ a: 1 });
```

Tuples & destructuring

Java

```java
record Pair<A,B>(A first, B second) {}
Pair<Integer,String> p = new Pair<>(1, "x");
int a = p.first(); String b = p.second();
```

TypeScript

```typescript
const pair: [number, string] = [1, "x"];
const [a, b] = pair;                    // tuple destructure
const { x, y } = point;                 // object destructure
const { x: px, y: py } = point;         // rename
const [head, ...rest] = xs;             // rest pattern
const ro = [1, 2] as const;             // readonly tuple
```

Mutability defaults

Java: `ArrayList`, `HashMap`, `HashSet` mutable; `List.of/Set.of/Map.of` are runtime-immutable.
TypeScript: All native collections are mutable at runtime. `readonly` / `Readonly<T>` / `ReadonlyMap` / `ReadonlySet` are *compile-time only* — they erase at runtime. For real runtime immutability use `Object.freeze` (shallow), `structuredClone` for deep copies, or libraries like `immer` / `immutable.js`.

7. Functions

Definition, default args, varargs

Java

```java
int add(int a, int b) { return a + b; }
int add(int a) { return add(a, 1); }    // overload
int sum(int... xs) { ... }
```

TypeScript

```typescript
function add(a: number, b: number = 1): number {
    return a + b;
}
function sum(...xs: number[]): number {
    return xs.reduce((t, x) => t + x, 0);
}
// Function overloads via signatures + impl:
function fmt(x: number): string;
function fmt(x: Date): string;
function fmt(x: number | Date): string { ... }
```

Arrow functions / closures

Java

```java
Function<Integer,Integer> sq = x -> x * x;
Predicate<String> nonEmpty = s -> !s.isEmpty();
list.forEach(System.out::println);
```

TypeScript

```typescript
const sq = (x: number) => x * x;
const nonEmpty = (s: string) => s.length > 0;
xs.forEach((x) => console.log(x));
// Arrow funcs: lexical `this`. Use `function` for own `this`.
```

Streams / pipelines

Java

```java
List<Integer> r = xs.stream()
    .filter(x -> x > 0)
    .map(x -> x * 2)
    .sorted()
    .toList();
int total = xs.stream().mapToInt(Integer::intValue).sum();
```

TypeScript

```typescript
const r = xs.filter(x => x > 0)
            .map(x => x * 2)
            .sort((a, b) => a - b);
const total = xs.reduce((a, x) => a + x, 0);
```

8. Classes, interfaces, types

Class with fields, ctor, methods

Java

```java
public class Point {
  private final double x, y;
  public Point(double x, double y) {
    this.x = x; this.y = y;
  }
  public double dist(Point o) {
    return Math.hypot(x - o.x, y - o.y);
  }
}
```

TypeScript

```typescript
class Point {
  // Param-property shorthand assigns + declares fields.
  constructor(
    public readonly x: number,
    public readonly y: number,
  ) {}
  dist(o: Point): number {
    return Math.hypot(this.x - o.x, this.y - o.y);
  }
}
```

Inheritance, abstract, interface

Java

```java
abstract class Shape {
  abstract double area();
}
interface Drawable { void draw(); }
class Circle extends Shape implements Drawable {
  private final double r;
  public Circle(double r) { this.r = r; }
  @Override double area() { return Math.PI*r*r; }
  @Override public void draw() { ... }
}
```

TypeScript

```typescript
abstract class Shape {
  abstract area(): number;
}
interface Drawable { draw(): void; }

class Circle extends Shape implements Drawable {
  constructor(private readonly r: number) { super(); }
  area(): number { return Math.PI * this.r ** 2; }
  draw(): void { ... }
}
// Single-class inheritance + multiple interface impl.
```

Static, getters/setters

Java

```java
public class Box {
  private int n;
  public int getN() { return n; }
  public void setN(int v) { n = v; }
  public static Box empty() { return new Box(); }
}
```

TypeScript

```typescript
class Box {
  private n = 0;
  get value(): number { return this.n; }
  set value(v: number) { this.n = v; }
  static empty(): Box { return new Box(); }
}
const b = new Box(); b.value = 5; console.log(b.value);
```

9. Type system: structural typing

Sidebar — structural vs nominal

Java types are **nominal**: `class A` and `class B` with identical members are unrelated. TypeScript types are **structural**: shape matches structure, not name. This is the single biggest mental shift coming from Java — it powers a lot of TS's flexibility (and surprises).

interface vs type alias

Java

```java
interface User { String name(); int age(); }
class Member implements User {
  public String name() { return "..."; }
  public int age() { return 0; }
}
// Must declare `implements`.
```

TypeScript

```typescript
interface User { name: string; age: number; }
// or:
type User = { name: string; age: number };

const m = { name: "x", age: 1, extra: true };
const u: User = m;          // OK - has all required props

// `interface` is open (mergeable). `type` is closed.
// Use interface for object/class shapes, type for unions.
```

Unions, literals, narrowing

Java

```java
// Closest analogs: sealed interfaces, enums.
sealed interface Status permits Active, Banned {}
record Active() implements Status {}
record Banned(String reason) implements Status {}
```

TypeScript

```typescript
type Status = "active" | "banned" | "pending";   // literals
type Result<T> = { ok: true; value: T }
               | { ok: false; error: Error };

function handle(r: Result<string>) {
  if (r.ok) {
    r.value;     // narrowed to { ok:true; value:string }
  } else {
    r.error;     // narrowed
  }
}
```

Type guards & assertion

Java

```java
if (s instanceof Member m) { use(m); }
// Pattern match (Java 16+)
```

TypeScript

```typescript
if (typeof x === "string") { ... }       // primitive
if (x instanceof Date)     { ... }       // class
if ("foo" in obj)          { ... }       // property test

function isUser(o: unknown): o is User {  // user-defined
  return typeof o === "object" && o !== null
      && "name" in o && typeof (o as any).name === "string";
}

const v = data as User;            // unsafe cast (avoid)
```

10. Generics / type parameters

Generic class & function

Java

```java
class Box<T> {
  private final T value;
  public Box(T v) { this.value = v; }
  public T get() { return value; }
}
<T extends Comparable<T>> T max(T a, T b) {
    return a.compareTo(b) >= 0 ? a : b;
}
List<? extends Number> ns;
```

TypeScript

```typescript
class Box<T> {
  constructor(public readonly value: T) {}
  get(): T { return this.value; }
}
function max<T extends { compareTo(o: T): number }>(
  a: T, b: T,
): T {
  return a.compareTo(b) >= 0 ? a : b;
}
// Bivariance & variance: TS doesn't model wildcards, but
// `readonly T[]` is roughly `List<? extends T>`.
```

Utility / mapped types

Java

```java
// No equivalent. You hand-write each variant class.
```

TypeScript

```typescript
type User = { name: string; age: number; email: string };
type PartialUser = Partial<User>;       // all optional
type ReadOnlyUser = Readonly<User>;
type Email       = Pick<User, "email">;
type NoEmail     = Omit<User, "email">;
type RequiredU   = Required<PartialUser>;
type Keys        = keyof User;          // "name"|"age"|...
// Mapped types compile to runtime-erased types.
```

11. Error handling

Try / catch / finally

Java

```java
try (var f = Files.newBufferedReader(p)) {
    return f.readLine();
} catch (IOException e) {
    log.error("read failed", e);
    throw new MyException("oops", e);
} finally { cleanup(); }
```

TypeScript

```typescript
try {
    return await readFile(p, "utf8");
} catch (e) {
    // Caught value type is `unknown` (TS 4.4+).
    if (e instanceof Error) {
        log.error(`read failed: ${e.message}`);
    }
    throw new MyError("oops", { cause: e });
} finally { cleanup(); }
```

Custom error

Java

```java
public class NotFoundException extends RuntimeException {
  public NotFoundException(String m) { super(m); }
}
throw new NotFoundException("user " + id);
```

TypeScript

```typescript
class NotFoundError extends Error {
  constructor(message: string) {
    super(message);
    this.name = "NotFoundError";
  }
}
throw new NotFoundError(`user ${id}`);
```

12. Modules / packages / imports

Imports / exports

Java

```java
package com.acme.svc;
import java.util.List;
import static com.acme.Utils.foo;
```

TypeScript

```typescript
import { foo, bar } from "./utils";              // named
import defaultExport from "./mod";               // default
import * as utils from "./utils";                // namespace
import type { User } from "./types";             // type-only

export const x = 1;
export function f() { ... }
export default class C { ... }
export type { User };
// "./utils.js" extension required in NodeNext / ESM mode.
```

13. Concurrency: Promises & async/await

Async functions

Java

```java
CompletableFuture<String> fut =
  CompletableFuture.supplyAsync(() -> fetch())
    .thenApply(String::trim)
    .exceptionally(e -> "fallback");
String r = fut.get();
```

TypeScript

```typescript
async function run(): Promise<string> {
    try {
        const raw = await fetchData();
        return raw.trim();
    } catch {
        return "fallback";
    }
}
const r = await run();        // top-level await in modules
```

Combinators & parallel

Java

```java
CompletableFuture<Void> all = CompletableFuture.allOf(f1, f2);
CompletableFuture<Object> any = CompletableFuture.anyOf(f1, f2);
```

TypeScript

```typescript
const [a, b] = await Promise.all([p1, p2]);     // all OK
const r = await Promise.race([p1, p2]);
const settled = await Promise.allSettled([p1,p2]);
const first  = await Promise.any([p1, p2]);     // any FULFILL
```

Single-threaded event loop

TypeScript runs on JS engines, which are **single-threaded** by default. `async/await` schedules continuations on the event loop — no preemption, no shared-memory races on JS objects. For CPU-heavy work, spawn `Worker` threads (browser) or `worker_threads` (Node), which have isolated memory and communicate via `postMessage`. This is a fundamental shift from Java's preëmptive threading + shared mutable state model.

14. Idiom appendix

Equality, null, undefined

Java

```java
if (s == null) { ... }              // identity
if (a.equals(b)) { ... }            // value
String x = (s != null) ? s : "default";
```

TypeScript

```typescript
if (s === null) { ... }             // strict equality
if (s == null)  { ... }             // null OR undefined
// Always prefer === / !== - `==` does coercion.
const x = s ?? "default";           // null/undefined only
const y = s || "default";           // ALSO falsy: "", 0, false
```

Common gotchas (TS coming from Java)

- **Types erase at runtime.** `instanceof T` works only for classes; you can't `instanceof MyInterface`. Use type guards or a `kind` discriminator.
- **this binding.** Arrow functions capture `this` lexically; `function` expressions don't. Methods passed as callbacks lose `this` unless bound.
- **undefined vs null.** Both exist. Default uninitialized = `undefined`. Prefer one consistently; many codebases ban `null`.
- **== is coercion.** `0 == ""` is true. Always `===`/`!==`.
- **Array holes.** `[1,,3]` has a sparse slot; `map` skips it. Avoid.
- **any vs unknown.** `any` disables type-checking. `unknown` forces narrowing before use — prefer it.
- **strict mode.** Always set `"strict": true` in `tsconfig.json`; otherwise nullability checks are off.
- **Numbers are floats.** No int. `0.1 + 0.2 !== 0.3`. Use `bigint` for exact integers, libraries for decimal money math.
- **Tooling.** `tsc` (compiler), `tsx`/`ts-node` (run), `eslint`, `prettier`, `vitest`/`jest`, `tsd`/`expect-type` for type-level tests.